TEST

Q1. Product of Array Except Self Given an integer array nums, return an array answer such that answer[i] is equal to the product of all elements of nums except nums[i]. You must solve it in O(n) time and without using division.
Example: Input: [1,2,3,4]
Output: [24,12,8,6]

```
public class Main {
    public static int[] productArray(int[] nums) {

        int n = nums.length;
        int[] res = new int[n];

        int leftProduct = 1;
        for (int i = 0; i < n; i++) {
            res[i] = leftProduct;
            leftProduct = leftProduct * nums[i];
        }

        int rightProduct = 1;
        for (int i = n - 1; i >= 0; i--) {
            res[i] = res[i] * rightProduct;
            rightProduct = rightProduct * nums[i];
        }

        return res;
    }

    public static void main(String[] args) {
        int[] nums = {1, 2, 3, 4};
        int[] answer = productExceptSelf(nums);
        for (int i = 0; i < res.length; i++) {
            System.out.print(answer[i] + " ");
        }
    }
}
```

Q2. Longest Consecutive Sequence Given an unsorted integer array nums, return the length of the longest consecutive elements sequence.
Your solution must run in O(n) time.
Example: Input: [100,4,200,1,3,2]
Output: 4
Explanation: The longest consecutive sequence is [1,2,3,4].

```
import java.util.*;
public class Main {
    public static int longestConsecutive(int[] nums) {
        HashSet<Integer> set = new HashSet<>();
        for (int num : nums) {
            set.add(num);
        }


        int maxLength = 0;
        for (int num : set) {
                if (!set.contains(num - 1)) {
                int currentNum = num;
                int length = 1;
                while (set.contains(currentNum + 1)) {
                    currentNum++;
                    length++;
                }
                maxLength = Math.max(maxLength, length);
            }
        }

        return maxLength;
    }

    public static void main(String[] args) {
        int[] nums = {100, 4, 200, 1, 3, 2};
        int result = longestConsecutive(nums);
        System.out.println("Output: " + result);
    }
}
```

Q3. 3Sum
Given an integer array nums, return all unique triplets [nums[i], nums[j], nums[k]] such that i, j, and k are distinct and nums[i] + nums[j] + nums[k] == 0.
The solution must not contain duplicate triplets.
Example:
Input: [-1,0,1,2,-1,-4]
Output: [[-1,-1,2],[-1,0,1]]

```
import java.util.*;
public class Main {
    public static List<List<Integer>> threeSum(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        Arrays.sort(nums);
        for (int i = 0; i < nums.length - 2; i++) {
            if (i > 0 && nums[i] == nums[i - 1]) {
                continue;
            }

            int left = i + 1;
            int right = nums.length - 1;
            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];
                if (sum == 0) {
                        result.add(Arrays.asList(nums[i],nums[left],nums[right]));
                    left++;
                    right--;

                    while (left < right &&
                            nums[left] == nums[left - 1]) {
                        left++;
                    }
                    while (left < right &&
                            nums[right] == nums[right + 1]) {
                        right--;
                    }

                }
                else if (sum < 0) {
                    left++;
                }
                else {
                    right--;
                }
            }
```

```
        }

        return result;
    }

    public static void main(String[] args) {
        int[] nums = {-1, 0, 1, 2, -1, -4};
        List<List<Integer>> result = threeSum(nums);
        System.out.println(result);
    }
}
```

Q4. Daily Temperatures Given an array temperatures, return an array where answer[i] tells you how many days you have to wait after the i-th day to get a warmer temperature. If no future day exists, return 0.
Example:
Input: [73,74,75,71,69,72,76,73]
Output: [1,1,4,2,1,1,0,0]

```
import java.util.*;
public class Main {
    public static int[] dailyTemperatures(int[] temperatures) {
        int n = temperatures.length;
        int[] result = new int[n];

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (temperatures[j] > temperatures[i]) {
                    result[i] = j - i;
                    break;
                }
            }
        }

        return result;
    }
    public static void main(String[] args) {
        int[] temperatures = {73, 74, 75, 71, 69, 72, 76, 73};
        int[] ans = dailyTemperatures(temperatures);

        System.out.println(Arrays.toString(ans));
    }
}
```

Q5. Kth Largest Element in an Array Given an integer array nums and an integer k, return the k-th largest element in the array.
Example:
Input: nums = [3,2,1,5,6,4]
k = 2
Output: 5

```
import java.util.*;
public class Main {
    public static int kthLargest(int[] nums, int k) {
        Arrays.sort(nums);
        int n = nums.length;
        return nums[n - k];
    }
    public static void main(String[] args) {
        int[] nums = {3, 2, 1, 5, 6, 4};
        int k = 2;
        int result = kthLargest(nums, k);
        System.out.println("Kth Largest Element: " + result);
    }
}
```

Q7. Course Schedule II There are numCourses courses labeled from 0 to numCourses - 1. prerequisites[i] = [a,b] means you must complete course b before taking course a. Return an ordering of courses you should take to finish all courses. If it is impossible, return an empty array.
Example:
Input: numCourses = 4 prerequisites = [[1,0],[2,0],[3,1],[3,2]]
Output: [0,2,1,3]

```
import java.util.*;
public class Main {
    public static int[] findOrder(int numCourses, int[][] prerequisites) {
        int[] count = new int[numCourses];
        for (int[] p : prerequisites) {
            count[p[0]]++;
        }


        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < numCourses; i++) {
            if (count[i] == 0) {
                queue.add(i);
            }
        }
        int[] result = new int[numCourses];
        int index = 0;
        while (!queue.isEmpty()) {
            int course = queue.remove();
            result[index] = course;
            index++;
            for (int[] p : prerequisites) {
                if (p[1] == course) {
                    count[p[0]]--;
                    if (count[p[0]] == 0) {
                        queue.add(p[0]);
                    }
                }
            }
        }
        if (index == numCourses) {
            return result;
        }
        return new int[0];
    }
```

```
    public static void main(String[] args) {
        int numCourses = 4;
        int[][] prerequisites = { {1, 0}, {2, 0}, {3, 1},{3, 2} };
        int[] result = findOrder(numCourses, prerequisites);
        System.out.println(Arrays.toString(result));
    }
}
```

Q8. Lowest Common Ancestor of a Binary Tree
Given the root of a binary tree and two nodes p and q, find their lowest common ancestor. Example:
Input Tree:
3
**/ **
5 1
**/ \\ / **
6 2 0 8
**/ **
7 4
p = 5, q = 1
Output: 3

```
public class Main {
    static class TreeNode {
        int val;
        TreeNode left, right;
        TreeNode(int val) {
            this.val = val;
        }
    }
    public static TreeNode LCA(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null) {
            return null;
        }
        if (root == p || root == q) {
            return root;
        }
        TreeNode left = LCA(root.left, p, q);
        TreeNode right = LCA(root.right, p, q);
        if (left != null && right != null) {
            return root;
        }
        if (left != null) {
            return left;
        }
        return right;
    }
    public static void main(String[] args) {
        TreeNode root = new TreeNode(3);
        root.left = new TreeNode(5);
        root.right = new TreeNode(1);
```

```
        root.left.left = new TreeNode(6);
        root.left.right = new TreeNode(2);

        root.right.left = new TreeNode(0);
        root.right.right = new TreeNode(8);

        TreeNode p = root.left;
        TreeNode q = root.right;
        TreeNode answer = LCA(root, p, q);
        System.out.println("LCA = " + answer.val);
    }
}
```

Q10. Longest Increasing Subsequence Given an integer array nums, return the length of the longest strictly increasing subsequence.
Example:
Input: [10,9,2,5,3,7,101,18]
Output: 4
Example subsequence: [2,3,7,101]

```
import java.util.*;
public class Main {
    public static int lengthOfLIS(int[] nums) {
        int n = nums.length;
        int[] dp = new int[n];
        Arrays.fill(dp, 1);
        int answer = 1;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[i] > nums[j]) {
                    dp[i] = Math.max(dp[i], dp[j] + 1);
                }
            }
            result = Math.max(answer, dp[i]);
        }

        return result;
    }
    public static void main(String[] args) {
        int[] nums = {10, 9, 2, 5, 3, 7, 101, 18};
        int ans = lengthOfLIS(nums);
        System.out.println(ans);
    }
}
```